

Lógica Computacional 2024/2025 - TP2

Grupo 6

- Cláudia Faria, a105531
- Patrícia Bastos, a102502

```
In [ ]: !pip install pysmt
!pysmt-install --z3

from z3 import *
import random
```

Exercício 1

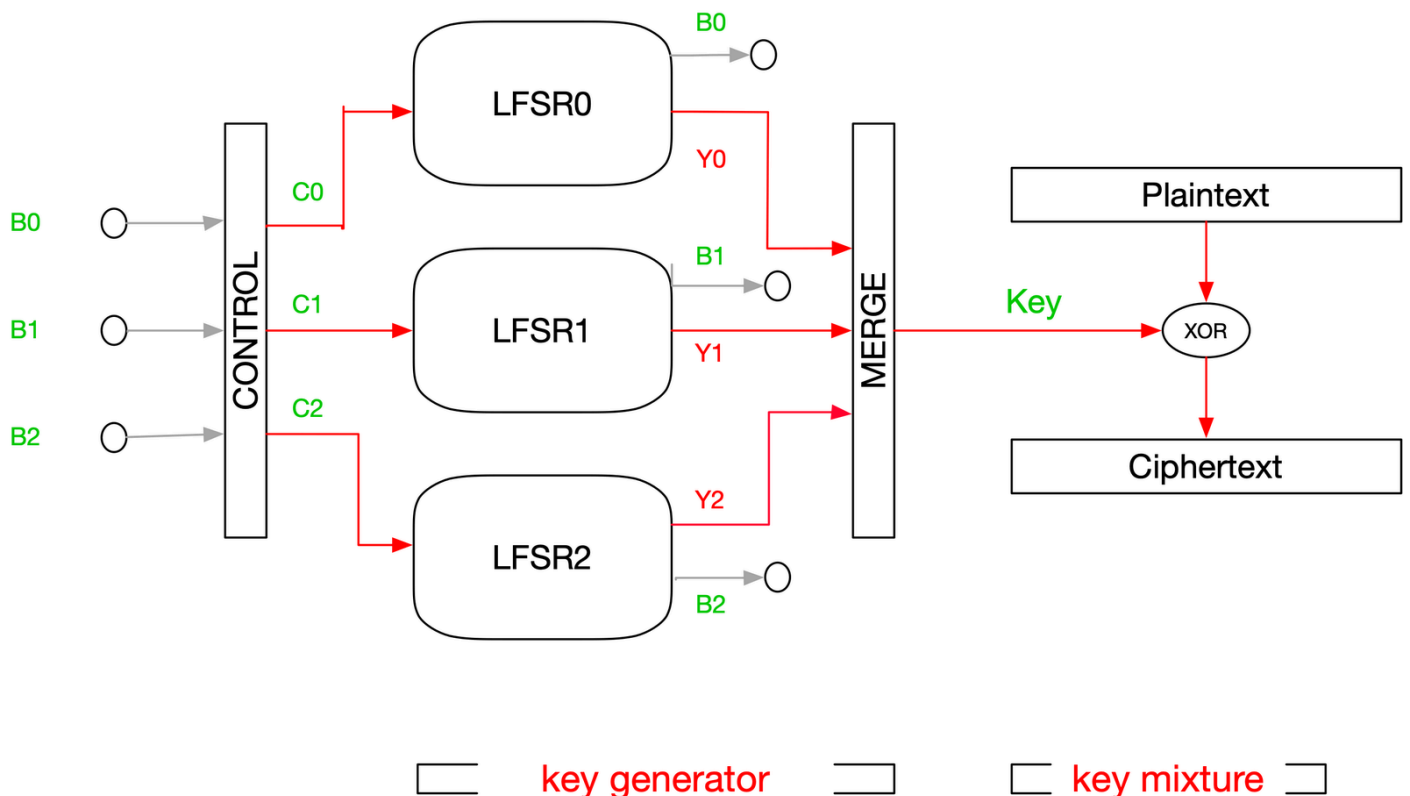
Enunciado

Considere a descrição da cifra A5/1. Pretende-se

- Definir e codificar, em Z3 e usando o tipo BitVec para modelar a informação, uma FSM que descreva o gerador de chaves.
- Considere as seguintes eventuais propriedades de erro:
 - ocorrência de um "burst" 0^t (t zeros) que ocorre em 2^t passos ou menos.
 - ocorrência de um "burst" de tamanho t que repete um "burst" anterior no mesmo output em $2^{(t/2)}$ passos ou menos.

Tente codificar estas propriedades e verificar se são acessíveis a partir de um estado inicial aleatoriamente gerado.

Resolução



Começamos por definir as seguintes funções:

- Função **declare(i)**, que declara o estado do sistema (os 3 LFSR, a key e a posição do último bit colocado na key) em cada instante i ;
- Função **init(state)**, que gera estados iniciais do sistema com valores aleatórios para cada LFSR;
- Função **majority(a, b, c)** que retorna o bit majoritário entre três bits dados;
- Função **proximo_estado(s)**, que calcula o próximo estado do sistema a partir do estado atual;
- Função **trans(curr, prox)**, que define a transição entre o estado atual e o próximo estado.

A função **proximo_estado** calcula o *clocking bit* de cada LSFR (no LFSR 0, é o bit de índice 8 e, nos LFSRs 1 e 2, é o bit de índice 10). A partir destes 3 bits, chama a função **majority** e calcula o *bit majoritário*.

A seguir, a função atualiza cada LSFR. Na cifra A5/1, cada LSFR é deslocado apenas se o seu *bit de controlo* é o *bit majoritário*, bit que calculamos anteriormente. Se o seu *bit de controlo* for diferente, então não é deslocado.

Posteriormente, construímos o novo bit através da operação *merge*. Neste caso, calculamos o *XOR* dos bits mais significativos de cada LSFR.

No final, atualizamos a posição atual no vetor dos *Key Bits* e atualizamos o próprio vetor.

```
In [ ]: def declare(i):
    return {
        'LFSR_0': BitVec(f'LFSR_0_{i}', 19),
        'LFSR_1': BitVec(f'LFSR_1_{i}', 22),
        'LFSR_2': BitVec(f'LFSR_2_{i}', 23),
        'keys': BitVec(f'keys_{i}', 64), # Vetor de 64 bits
        'keys_pos': BitVec(f'keys_pos_{i}', 7) # Posição atual no vetor (0-63), representada por 7 bits
    }

def init(state):
    z0 = random.getrandbits(19)
    z1 = random.getrandbits(22)
    z2 = random.getrandbits(23)

    return And(
        state['LFSR_0'] == BitVecVal(z0, 19),
        state['LFSR_1'] == BitVecVal(z1, 22),
        state['LFSR_2'] == BitVecVal(z2, 23),
        state['keys'] == BitVecVal(0, 64), # Inicializado com zero
        state['keys_pos'] == BitVecVal(0, 7) # Posição inicializada com zero
    )

def majority(a, b, c):
    return (a & b) | (b & c) | (c & a)

def proximo_estado(s):
    c0 = Extract(8, 8, s['LFSR_0'])
    c1 = Extract(10, 10, s['LFSR_1'])
    c2 = Extract(10, 10, s['LFSR_2'])

    maj = majority(c0, c1, c2)

    # Desloca os LFSR e atualiza usando uma função XOR
    updated_LFSR_0 = If(c0 == maj,
        Concat(Extract(17, 0, s['LFSR_0']),
            Extract(18, 18, s['LFSR_0']) ^ Extract(17, 17, s['LFSR_0']) ^
            Extract(16, 16, s['LFSR_0']) ^ Extract(13, 13, s['LFSR_0'])),
        s['LFSR_0'])

    updated_LFSR_1 = If(c1 == maj,
        Concat(Extract(20, 0, s['LFSR_1']),
            Extract(21, 21, s['LFSR_1']) ^ Extract(20, 20, s['LFSR_1'])),
        s['LFSR_1'])

    updated_LFSR_2 = If(c2 == maj,
        Concat(Extract(21, 0, s['LFSR_2']),
            Extract(22, 22, s['LFSR_2']) ^ Extract(21, 21, s['LFSR_2']) ^
            Extract(20, 20, s['LFSR_2']) ^ Extract(7, 7, s['LFSR_2'])),
        s['LFSR_2'])

    # XOR dos bits mais significativos
    updated_bit = Extract(18, 18, updated_LFSR_0) ^ Extract(21, 21, updated_LFSR_1) ^ Extract(22, 22, updated_LFSR_2)

    # Atualiza a posição do vetor
    updated_pos = s['keys_pos'] + 1

    # Atualiza o vetor inserindo o novo bit se a posição for menor que 63
    updated_keys = If(ULT(s['keys_pos'], BitVecVal(63, 7)),
        s['keys'] | (ZeroExt(63, updated_bit) << ZeroExt(57, s['keys_pos'])),
        s['keys'])

    return {
        'LFSR_0': updated_LFSR_0,
        'LFSR_1': updated_LFSR_1,
        'LFSR_2': updated_LFSR_2,
        'keys': updated_keys,
        'keys_pos': updated_pos
    }

def trans(curr, prox):
    proximo = proximo_estado(curr)
    return And(
        prox['LFSR_0'] == proximo['LFSR_0'],
        prox['LFSR_1'] == proximo['LFSR_1'],
        prox['LFSR_2'] == proximo['LFSR_2'],
        prox['keys'] == proximo['keys'],
        prox['keys_pos'] == proximo['keys_pos']
    )
```

Definimos as propriedades de erro da alínea b) em duas funções.

- A função **check_burst_zeros(bits, t, pos)** verifica se há um burst de zeros contínuos de comprimento 't' no output inicializado.
- A função **check_burst_repetido(bits, t, pos)** verifica se há um burst de bits repetidos de comprimento 't' no output inicializado.

```
In [ ]: def check_burst_zeros(bits, t, pos):
    max_pos = pos.as_long() # Valor máximo da posição
    # Itera de 0 até max_pos - t + 1 (todas as posições possíveis), extrai sequência de t bits e adiciona a lista
    # Resultado de 0r sobre a lista verifica se alguma sequência tem t bits zero
    return Or([Extract(i + t - 1, i, bits) == 0 for i in range(max_pos - t + 1)])

def check_burst_repetido(bits, t, pos):
    max_pos = pos.as_long() # Valor máximo da posição
    # Resultado de 0r sobre a lista verifica se alguma é repetida
    return Or([And(Extract(i + t - 1, i, bits) == Extract(j + t - 1, j, bits), # Extrai sequência de t bits
        i != j, # Sequências devem ser diferentes
        j - i <= 2 ** (t // 2)) # Distância entre as sequências deve ser no máximo 2 ** (t // 2)
        for i in range(max_pos - t + 1) # 0 até max_pos - t + 1 (todas as posições possíveis)
        for j in range(i + 1, min(i + 2 ** (t // 2) + 1, max_pos - t + 1))] # i + 1 até min(i + 2 ** (t // 2) + 1, max_pos - t + 1)
        # (posições que estão a uma distância de no máximo 2 ** (t // 2) de i)
```

A função **gera_traco(declare, init, trans, k, t)** começa por declarar, inicializar o sistema e o Solver e definir a evolução do sistema em cada transição. É criada uma lista vazia *detectado*, onde serão colocados todos os estados em que for detectada uma propriedade de erro. Em cada estado, verifica-se as propriedades de erro e, para visualização, é imprimido cada LFSR, o vetor *Key Bits* para cada estado e a ocorrência de propriedade de erro, se for o caso. No final, se tiver sido detectada alguma propriedade de erro, os estados em que ocorreu são listados.

```
In [ ]: def gera_traco(declare, init, trans, k, t):
    states = [declare(i) for i in range(k + 1)] # Declara o estado do sistema
    solver = Solver() # Inicializa o Solver
    solver.add(init(states[0])) # Inicializa aleatoriamente o sistema

    for i in range(k):
        solver.add(trans(states[i], states[i + 1])) # Define evolução do sistema

    detectado = []
    for i, state in enumerate(states):
        # Verifica se o solver é satisfatório antes de chamar o modelo
        if solver.check() == sat:
            model = solver.model()
            LFSR_0_val = model[state['LFSR_0']].as_long() # as_long converte BitVec para número inteiro
            LFSR_1_val = model[state['LFSR_1']].as_long()
            LFSR_2_val = model[state['LFSR_2']].as_long()
            keys_val = model[state['keys']].as_long()
            pos_val = model[state['keys_pos']].as_long()

            # Imprime
            print(f"\nEstado {i}:")
            print("-" * 40)
            LFSR_0_bin = bin(LFSR_0_val)[2:].zfill(19) # bin converte número inteiro para binário
            LFSR_1_bin = bin(LFSR_1_val)[2:].zfill(22) # remove '0b' , zfill() preenche a string com zeros à esquerda até atingir o comprimento
            LFSR_2_bin = bin(LFSR_2_val)[2:].zfill(23)
            keys_bin = bin(keys_val)[2:].zfill(64)
            print(f"LFSR 0: {LFSR_0_bin}")
            print(f"LFSR 1: {LFSR_1_bin}")
            print(f"LFSR 2: {LFSR_2_bin}")
            print(f"Output ({pos_val} bits preenchidos): {keys_bin}")

            # Verifica as propriedades de erro
            if pos_val > 0: # Não se verifica o estado inicial
                burst_zeros = check_burst_zeros(state['keys'], t, BitVecVal(pos_val, 7))
                burst_repetido = check_burst_repetido(state['keys'], t, BitVecVal(pos_val, 7))

                if solver.check(burst_zeros) == sat:
                    print("*** Burst de zeros detectado neste estado ***")
                    detectado.append(i)
                if solver.check(burst_repetido) == sat:
                    print("*** Burst repetido detectado neste estado ***")
                    detectado.append(i)
            else:
                print(f"> Erro ao gerar estado {i}.")

    if detectado:
        print(f"\nPropriedades de erro detectadas nos estados: {sorted(set(detectado))}")
    else:
        print(f"\n> Nenhuma propriedade de erro detectada para burst de tamanho {t}.")
```

Por fim, podemos testar o código para o número de iterações e o tamanho de burst que desejarmos. Para que seja possível visualizar os Key Bits todos, o número de iterações deve ser menor que 64.

```
In [ ]: k = int(input("Forneça um número de iterações: "))
print(f"Calcular com número de iterações = {k}...\n")
t = int(input("Forneça um tamanho de burst: "))
print(f"Calcular com tamanho = {t}...\n")

gera_traco(declare, init, trans, k, t)
```

Forneça um tamanho de burst: 4
Calcular com tamanho = 4...

```
LFSR 0: 0111100101100001101
LFSR 1: 0111001010100000000111
```

Estado 25:

```
LFSR 0: 0000100001000101010  
LFSR 1: 0000111100101111110000  
LFSR 2: 10000000100100100111101  
Output (37 bits preenchidos): 000000000000000000000000000010011110110001110101110111110111111  
*** Burst repetido detectado neste estado ***
```

```
Estado 38:
-----
LFSR 0: 0001000010001010100
LFSR 1: 0001111001011111100000
LFSR 2: 00000001001001001111011
Output (38 bits preenchidos): 000000000000000000000000100111101100011101011101111101111111
*** Burst repetido detectado neste estado ***
```

```
Estado 39:
-----
LFSR 0: 0010000100010101000
LFSR 1: 0001111001011111100000
LFSR 2: 00000010010010011110110
Output (39 bits preenchidos): 000000000000000000000000100111101100011101011101111101111111
*** Burst repetido detectado neste estado ***
```

```
Estado 40:
-----
LFSR 0: 0010000100010101000
LFSR 1: 0011110010111111000000
LFSR 2: 00000100100100111101101
Output (40 bits preenchidos): 000000000000000000000000100111101100011101011101111101111111
*** Burst repetido detectado neste estado ***
```

Propriedades de erro detectadas nos estados: [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40]

Interpretação dos resultados

Diferentes testes apoiam a nossa interpretação das propriedades de erro. Experimente os seguintes testes:

```
In [ ]: print("-" * 100)
        print("\nPara k = 30 e t = 4:")
        gera_traco(declare, init, trans, 30, 4)

        print("-" * 100)
        print("\nPara k = 30 e t = 20:")
        gera_traco(declare, init, trans, 30, 20)

        print("-" * 100)
        print("\nPara k = 64 e t = 4:")
        gera_traco(declare, init, trans, 64, 4)

        print("-" * 100)
        print("\nPara k = 64 e t = 20:")
        gera_traco(declare, init, trans, 64, 20)
```

Estado 0:

Estado 1:

Estado 2:

Estado 3:

Estado 4:

Estado 5:

Estado 6:

Estado 7:

Estado 8:

Estado 9:

Estado 10:

Estado 11:

Estado 12:

Estado 13:

```
LFSR 0: 0011011011110011011
LFSR 1: 0100011101010011101101
```


Estado 26:

Estado 27:

Estado 28:

Estado 29:

Estado 30:

Propriedades de erro detectadas nos estados: [10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30]

Estado 0:

Estado 1:

Estado 2:

Estado 3:

Estado 4:

Estado 5:

Estado 6:

Estado 7:

```
LFSR 0: 0011101111101101100
LFSR 1: 11011011110000000111011
```

[illegible]

Estado 6:

Estado 7:

Estado 8:

Estado 9:

Estado 10:

Estado 11:

Estado 12:

Estado 13:

Estado 14:

Estado 15:

Estado 16:

Estado 17:

[illegible]

[illegible]

Estado 41:

LFSR 0: 1100010011001010100
LFSR 1: 1010101101000101000011
LFSR 2: 11101001111111001110110
Output (41 bits preenchidos): 0000000000000000000011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 42:

LFSR 0: 1000100110010101001
LFSR 1: 0101011010001010000111
LFSR 2: 11101001111111001110110
Output (42 bits preenchidos): 0000000000000000000011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 43:

LFSR 0: 0001001100101010011
LFSR 1: 1010110100010100001111
LFSR 2: 11101001111111001110110
Output (43 bits preenchidos): 0000000000000000000011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 44:

LFSR 0: 0010011001010100110
LFSR 1: 0101101000101000011111
LFSR 2: 1101001111110011101101
Output (44 bits preenchidos): 000000000000000000010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 45:

LFSR 0: 0100110010101001100
LFSR 1: 1011010001010000111111
LFSR 2: 1101001111110011101101
Output (45 bits preenchidos): 000000000000000000010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 46:

LFSR 0: 1001100101010011000
LFSR 1: 0110100010100001111111
LFSR 2: 10100111111100111011011
Output (46 bits preenchidos): 000000000000000000010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 47:

LFSR 0: 0011001010100110001
LFSR 1: 1101000101000011111111
LFSR 2: 01001111111001110110111
Output (47 bits preenchidos): 0000000000000000010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 48:

LFSR 0: 0011001010100110001
LFSR 1: 1010001010000111111110
LFSR 2: 1001111111001110110110
Output (48 bits preenchidos): 0000000000000000010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 49:

LFSR 0: 0110010101001100011
LFSR 1: 1010001010000111111110
LFSR 2: 0011111110011101101101
Output (49 bits preenchidos): 0000000000000001010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 50:

LFSR 0: 1100101010011000111
LFSR 1: 0100010100001111111101
LFSR 2: 0011111110011101101101
Output (50 bits preenchidos): 0000000000000011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 51:

LFSR 0: 1001010100110001110
LFSR 1: 1000101000011111111011
LFSR 2: 0011111110011101101101
Output (51 bits preenchidos): 0000000000000011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 52:

LFSR 0: 0010101001100011100
LFSR 1: 000101000011111110111

LFSR 2: 01111111001110110111010
Output (52 bits preenchidos): 000000000000011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 53:

LFSR 0: 0101010011000111001
LFSR 1: 001010000111111101110
LFSR 2: 1111110011101101110101
Output (53 bits preenchidos): 0000000000010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 54:

LFSR 0: 1010100110001110010
LFSR 1: 001010000111111101110
LFSR 2: 1111100111011011101011
Output (54 bits preenchidos): 0000000000010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 55:

LFSR 0: 1010100110001110010
LFSR 1: 0101000011111111011100
LFSR 2: 1111001110110111010110
Output (55 bits preenchidos): 0000000000010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 56:

LFSR 0: 1010100110001110010
LFSR 1: 101000011111110111001
LFSR 2: 11110011101101110101100
Output (56 bits preenchidos): 0000000010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 57:

LFSR 0: 0101001100011100100
LFSR 1: 101000011111110111001
LFSR 2: 11100111011011101011000
Output (57 bits preenchidos): 0000000010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 58:

LFSR 0: 0101001100011100100
LFSR 1: 010000111111101110011
LFSR 2: 11001110110111010110001
Output (58 bits preenchidos): 0000001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 59:

LFSR 0: 0101001100011100100
LFSR 1: 1000011111111011100111
LFSR 2: 10011101101110101100011
Output (59 bits preenchidos): 0000001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 60:

LFSR 0: 0101001100011100100
LFSR 1: 0000111111110111001111
LFSR 2: 001110110111011000111
Output (60 bits preenchidos): 0000001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 61:

LFSR 0: 1010011000111001001
LFSR 1: 0000111111110111001111
LFSR 2: 01110110111010110001110
Output (61 bits preenchidos): 0001001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 62:

LFSR 0: 0100110001110010011
LFSR 1: 0001111111101110011110
LFSR 2: 11101101110101100011101
Output (62 bits preenchidos): 0011001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Estado 63:

LFSR 0: 0100110001110010011
LFSR 1: 0011111111011100111100
LFSR 2: 1101011101011000111011
Output (63 bits preenchidos): 0111001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***

*** Burst repetido detectado neste estado ***

Estado 64:

LFSR 0: 1001100011100100110
LFSR 1: 0111111110111001111000
LFSR 2: 10110111010110001110110
Output (64 bits preenchidos): 0111001010010011010010011110001001000111100000100110101000100100
*** Burst de zeros detectado neste estado ***
*** Burst repetido detectado neste estado ***

Propriedades de erro detectadas nos estados: [7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64]

Para k = 64 e t = 20:

Estado 0:

LFSR 0: 0111010100100100000
LFSR 1: 0110110000010000000101
LFSR 2: 000011110111110101101
Output (0 bits preenchidos): 00

Estado 1:

LFSR 0: 1110101001001000001
LFSR 1: 1101100000100000001011
LFSR 2: 0001111011111010111010
Output (1 bits preenchidos): 00

Estado 2:

LFSR 0: 1101010010010000011
LFSR 1: 1011000001000000010110
LFSR 2: 0001111011111010111010
Output (2 bits preenchidos): 00

Estado 3:

LFSR 0: 1010100100100000111
LFSR 1: 0110000010000000101101
LFSR 2: 0001111011111010111010
Output (3 bits preenchidos): 00100

Estado 4:

LFSR 0: 0101001001000001110
LFSR 1: 0110000010000000101101
LFSR 2: 0011110111110101110101
Output (4 bits preenchidos): 00100

Estado 5:

LFSR 0: 1010010010000011101
LFSR 1: 1100000100000001011011
LFSR 2: 0011110111110101110101
Output (5 bits preenchidos): 00100

Estado 6:

LFSR 0: 0100100100000111011
LFSR 1: 1000001000000010110110
LFSR 2: 0011110111110101110101
Output (6 bits preenchidos): 00100100

Estado 7:

LFSR 0: 1001001000001110111
LFSR 1: 0000010000000101101101
LFSR 2: 0011110111110101110101
Output (7 bits preenchidos): 001100100

Estado 8:

LFSR 0: 0010010000011101111
LFSR 1: 0000100000001011011010
LFSR 2: 0011110111110101110101
Output (8 bits preenchidos): 001100100

Estado 9:

LFSR 0: 0100100000111011110
LFSR 1: 0001000000010110110100
LFSR 2: 0011110111110101110101
Output (9 bits preenchidos): 001100100

Estado 10:

LFSR 0: 1001000001110111101
LFSR 1: 0010000000101101101000
LFSR 2: 01111101111101011101011
Output (10 bits preenchidos): 001001100100

Estado 11:

LFSR 0: 1001000001110111101
LFSR 1: 0100000001011011010000
LFSR 2: 1111101111101011101011
Output (11 bits preenchidos): 001001100100

Estado 12:

Estado 41:

LFSR 0: 111111011000111000
LFSR 1: 0110000011101111000
LFSR 2: 10100101001101110111
Output (41 bits preenchidos): 000000000000000000000000110101101011110011101101001001100100

Estado 42:

LFSR 0: 1111110110001110000
LFSR 1: 0110000011101111000
LFSR 2: 01001010011011101111
Output (42 bits preenchidos): 00000000000000000000000100000110101101011110011101101001001100100

Estado 43:

LFSR 0: 1111110110001110000
LFSR 1: 110000011101101110001
LFSR 2: 10010100110111011111
Output (43 bits preenchidos): 00000000000000000000000110000110101101011110011101101001001100100

Estado 44:

LFSR 0: 1111101100011100000
LFSR 1: 100000111011011100010
LFSR 2: 100101001101110110111
Output (44 bits preenchidos): 000000000000000000000001110000110101101011110011101101001001100100

Estado 45:

LFSR 0: 1111101100011100000
LFSR 1: 000001110110111000101
LFSR 2: 0010100110111011011110
Output (45 bits preenchidos): 000000000000000000000001110000110101101011110011101101001001100100

Estado 46:

LFSR 0: 1111101100011100000
LFSR 1: 000011101101110001010
LFSR 2: 0101001101101101111100
Output (46 bits preenchidos): 0000000000000000000000011110000110101101011110011101101001001100100

Estado 47:

LFSR 0: 11111011000111000001
LFSR 1: 0000111011011100010100
LFSR 2: 10100110111011011111001
Output (47 bits preenchidos): 0000000000000000000000011110000110101101011110011101101001001100100

Estado 48:

LFSR 0: 1110110001110000010
LFSR 1: 0001110110111000101000
LFSR 2: 01001101110110111110011
Output (48 bits preenchidos): 0000000000000000010111110000110101101011110011101101001001100100

Estado 49:

LFSR 0: 1101100011100000100
LFSR 1: 0011101101110001010000
LFSR 2: 10011011101101111100110
Output (49 bits preenchidos): 0000000000000000010111110000110101101011110011101101001001100100

Estado 50:

LFSR 0: 1011000111000001000
LFSR 1: 0111011011100010100000
LFSR 2: 10011011101101111100110
Output (50 bits preenchidos): 0000000000000000010111110000110101101011110011101101001001100100

Estado 51:

LFSR 0: 0110001110000010000
LFSR 1: 1110110111000101000001
LFSR 2: 00110111011011111001100
Output (51 bits preenchidos): 000000000000010010111110000110101101011110011101101001001100100

Estado 52:

LFSR 0: 1100011100000100000
LFSR 1: 1101101110001010000010
LFSR 2: 00110111011011111001100
Output (52 bits preenchidos): 000000000000010010111110000110101101011110011101101001001100100

Estado 53:

LFSR 0: 1000111000001000001
LFSR 1: 1011011100010100000100
LFSR 2: 001101110110111111001100
Output (53 bits preenchidos): 000000000000010010111110000110101101011110011101101001001100100

Estado 54:

LFSR 0: 1000111000001000001
LFSR 1: 0110111000101000001001
LFSR 2: 0110111011011110011000
Output (54 bits preenchidos): 00000000010010010111110000110101101011110011101101001001100100

Estado 55:

LFSR 0: 0001110000010000010
LFSR 1: 1101110001010000010011
LFSR 2: 01101110110111110011000
Output (55 bits preenchidos): 000000000110010010111110000110101101011110011101101001001100100

Estado 56:

```
-----
LFSR 0: 0001110000010000010
LFSR 1: 1011100010100000100110
LFSR 2: 11011101101111100110001
Output (56 bits preenchidos): 00000000011001001011111000001101011010111100111011001001100100
```

Estado 57:

```
-----
LFSR 0: 0011100000100000101
LFSR 1: 0111000101000001001101
LFSR 2: 11011101101111100110001
Output (57 bits preenchidos): 0000000101100100101111100000110101101011110011101101001001100100
```

Estado 58:

```
-----
LFSR 0: 0111000001000001011
LFSR 1: 0111000101000001001101
LFSR 2: 10111011011111001100010
Output (58 bits preenchidos): 0000001101100100101111100000110101101011110011101101001001100100
```

Estado 59:

```
-----
LFSR 0: 1110000010000010110
LFSR 1: 1110001010000010011011
LFSR 2: 10111011011111001100010
Output (59 bits preenchidos): 0000011101100100101111100000110101101011110011101101001001100100
```

Estado 60:

```
-----
LFSR 0: 1100000100000101101
LFSR 1: 1100010100000100110110
LFSR 2: 10111011011111001100010
Output (60 bits preenchidos): 0000111101100100101111100000110101101011110011101101001001100100
```

Estado 61:

```
-----
LFSR 0: 1000001000001011010
LFSR 1: 1000101000001001101100
LFSR 2: 10111011011111001100010
Output (61 bits preenchidos): 0001111101100100101111100000110101101011110011101101001001100100
```

Estado 62:

```
-----
LFSR 0: 0000010000010110101
LFSR 1: 0001010000010011011001
LFSR 2: 10111011011111001100010
Output (62 bits preenchidos): 0011111101100100101111100000110101101011110011101101001001100100
```

Estado 63:

```
-----
LFSR 0: 0000010000010110101
LFSR 1: 0010100000100110110010
LFSR 2: 01110110111110011000100
Output (63 bits preenchidos): 0011111101100100101111100000110101101011110011101101001001100100
```

Estado 64:

```
-----
LFSR 0: 0000100000101101011
LFSR 1: 0101000001001101100100
LFSR 2: 01110110111110011000100
Output (64 bits preenchidos): 0011111101100100101111100000110101101011110011101101001001100100
```

> Nenhuma propriedade de erro detectada para burst de tamanho 20.

Influência do parâmetro k (número de iterações)

- Se o número de passos k é pequeno, o sistema tem menos oportunidades de gerar um burst de zeros ou de repetir um padrão de tamanho t. Isso reduz a probabilidade de ocorrência de ambas as propriedades de erro.
- À medida que aumentamos k, o sistema tem mais iterações e portanto mais sequências possíveis de saída. Isto aumenta a probabilidade de ser detectado um burst de zeros ou um burst repetido no limite de passos definido.

Influência do parâmetro t (tamanho do burst)

- Se t é pequeno, o burst ocorre mais facilmente, uma vez que a repetição de um padrão curto de bits é mais provável do que um padrão comprido.
- À medida que aumentamos t, a ocorrência de um mesmo burst de zeros torna-se menos provável, assim como a repetição de um burst, uma vez que existem mais combinações possíveis para t bits.

Conclui-se que o aumento de k torna as propriedades de erro mais prováveis, enquanto que o aumento de t torna as propriedades de erro menos prováveis.